Blockchain Integration

Implementation Guide

Multi-Tenant E-Voting System · Phase-1 · Document 1 Corrections & Full Build Instructions

What this document covers

- All changes required to Document 1 based on the architectural corrections in Document 2
- Step-by-step implementation instructions for every blockchain component
- Recommended tech stack, tooling, and directory structure for the college project
- Smart contract code outline, frontend Web3 integration, and backend sync logic
- Gas management strategy for development and final demo

Section 1 — Corrections Required in Document 1

Document 2 identifies three places where Document 1 contains architectural assumptions that would compromise the security model of the blockchain voting system. Each correction is described below with the exact location in Document 1, what it currently says, what it should say, and why the change matters.

1.1 Ballot Casting Flow — Transaction Signing Responsibility

⚠ **Critical Change** This is the most important correction. The original flow implies the backend submits the vote transaction. This must change.

Location in Document 1: Section "The Casting of the Ballot and On-Chain Finality" — Steps 1–4 of the ballot casting table.

❌ Original (Incorrect)	✅ Corrected Version
❌ Original (Incorrect)	✅ Corrected Version
Step 1: Frontend requests a Merkle Proof from the backend. Step 4: "Smart contract verifies proof" — implied the backend calls the contract.	Step 1: Frontend requests Merkle Proof from backend. ✔ (unchanged) Step 2: MetaMask pops up — voter reviews and signs transaction. ✔ (unchanged)

❌ Original (Incorrect)	✅ Corrected Version
Step 5: Backend monitors the blockchain and updates DB.	Step 3: Signed Tx is broadcast directly to the blockchain by the frontend wallet — NOT the backend. Step 4: Smart contract enforces all verification on-chain. Step 5: Frontend receives Tx hash → sends it to backend API → backend saves to DB.

📌 **Why this matters** If the backend submits the vote, the private key must live on the server — making it technically identical to a centralized database. Any admin could fabricate votes. The blockchain provides no security guarantee in that model.

1.2 Double-Voting Prevention — Authority Must Move On-Chain

Location in Document 1: Section "The Casting of the Ballot" — the implied pre-ballot check in the backend before showing the ballot screen.

❌ Original Role	✅ Corrected Role
Backend performs a pre-ballot check to verify the user has not already voted. This is described as a guard before the ballot UI is shown.	The smart contract maintains an on-chain mapping of voted addresses. This is the authoritative check — the contract will revert the transaction if the address has already voted, regardless of what the backend says. The backend check becomes a UX hint only — it may grey-out the Vote button as a convenience, but it does not enforce the rule.

📌 **Why this matters** A compromised backend could skip the pre-ballot check and allow a voter to vote twice. The smart contract cannot be bypassed — it enforces the rule regardless of what the application layer does.

1.3 Transaction Hash Recording — Flow Direction Change

Location in Document 1: Section "The Casting of the Ballot" — Step 5: "Backend monitors the blockchain and updates DB when the Tx is finalized."

❌ Original Flow	✅ Corrected Flow
Backend "monitors" the blockchain to receive the Tx hash and update the database. This assumes the backend initiated the	Since the frontend wallet submits the transaction, the frontend receives the Tx hash from the wallet provider (ethers.js). The

❌ Original Flow	✅ Corrected Flow
transaction and is watching for its own submission.	frontend then sends this hash to the backend via a dedicated API endpoint (POST /api/votes/confirm). The backend stores: voter ID, election ID, wallet address, Tx hash, and timestamp. It uses this for the voter dashboard and audit trail only — the blockchain is still the source of truth.

1.4 Parts of Document 1 That Require No Changes

✅ **Confirmed Correct** The following sections in Document 1 are architecturally sound and need no modifications.

Section	Location	Status
Authentication & MFA Flow	Sections 1–5	Fully off-chain. JWT, OTP, user DB — no blockchain involvement.
Organization Registration	Super Admin Flow	Domain verification, role provisioning — off-chain. Correct.
Election Initialization	Org Admin Steps 1–5	Draft creation in relational DB. Correct.
Candidate Lifecycle	Admin Steps 1–4	Hybrid storage (DB + S3 + SHA-256 hash). Correct.
Merkle Tree Construction	Voter Eligibility Section	Root generation in backend, stored on-chain in contract. Correct.
Smart Contract Deployment	Transition to Immutability	Backend deploys contract, stores address. Correct.
Pre-Verification Flow	One-Time Pre-Verification	Voter ID checked before election day. Off-chain. Correct.
Result Committee Audit	Dual-Source Audit Flow	RC reads blockchain + DB for comparison. Correct.
Failure Handling	Technical Resilience Section	Gas errors, pending tx recovery. Correct in principle.

Section 2 — Recommended Technology Stack

The following tools are recommended specifically for a college project — chosen for free tier availability, strong documentation, and simplicity of local development.

2.1 Full Stack Summary

Layer	Tool	Notes
Smart Contracts	Solidity ^0.8.20	Industry standard. Compile with Hardhat.
Local Blockchain	Hardhat Network	Faster than Ganache, built-in debugging, scriptable.
Public Testnet	Sepolia (Ethereum)	Ethereum's current active testnet. Free ETH from faucets.
Contract Interaction	ethers.js v6	Lighter and cleaner API than web3.js.
Wallet	MetaMask (browser extension)	Standard wallet. Easy to demo. Works with Hardhat & Sepolia.
RPC Provider	Alchemy (free tier)	Needed to connect to Sepolia. 300M compute units/month free.
Contract Dev Tool	Hardhat + Hardhat-ignition	Compile, test, deploy, and debug contracts locally.
Backend	Node.js + Express (existing)	Backend deployment uses ethers.js server-side.
Frontend	React + ethers.js	Frontend uses ethers.js to call MetaMask and contracts.
Database	PostgreSQL (existing)	Stores election metadata, voter records, Tx hashes.
File Storage	AWS S3 / MinIO (existing)	Manifesto PDFs and candidate photos. No change needed.

2.2 Project Directory Structure

📁 **Directory Layout**

```
evoting-project/
```

```
├── contracts/              ← Solidity smart contracts
│   ├── VotingFactory.sol   ← Deploys individual election contracts
│   └── Election.sol        ← Core voting logic per election
├── scripts/
│   └── deploy.js           ← Hardhat deployment script
├── test/
│   └── Election.test.js    ← Contract unit tests
├── hardhat.config.js       ← Network config (local + Sepolia)
├── backend/
│   ├── routes/
│   │   ├── elections.js    ← Deploy contract, store address
│   │   └── votes.js        ← Receive Tx hash from frontend
│   └── services/
│       └── blockchain.js   ← ethers.js read-only calls
└── frontend/
    ├── services/
    │   └── web3.js         ← Wallet connection + contract calls
    └── pages/
        └── VotingPage.jsx  ← Ballot UI + MetaMask trigger
```

Section 3 — Smart Contract Implementation

The smart contract is the core of the system's integrity. It enforces all voting rules autonomously — no backend can override it once deployed.

3.1 Election.sol — Structure and Logic

Create this file at `contracts/Election.sol`. The contract receives all election parameters in its constructor and enforces them on every vote transaction.

contracts/Election.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract Election {

    // ── State variables ──────────────────────────────────
    address public admin;          // Backend wallet that deployed this
    uint256 public startTime;      // UNIX timestamp
    uint256 public endTime;        // UNIX timestamp
    bytes32 public eligibilityRoot;// Merkle root of authorized voters
    bool    public finalized;      // Locked after RC finalizes

    // Candidate IDs passed in at deploy (e.g., [1, 2, 3])
    uint256[] public candidateIds;

    // Maps candidateId → vote count
    mapping(uint256 => uint256) public voteCounts;
```

```
    // Maps wallet address → has voted (prevents double voting)
    mapping(address => bool) public hasVoted;

    // ── Events ──────────────────────────────────────────────
    event VoteCast(address indexed voter, uint256 candidateId, uint256 timestamp);
    event ElectionFinalized(uint256 timestamp);

    // ── Constructor ─────────────────────────────────────────
    constructor(
        uint256[] memory _candidateIds,
        uint256 _startTime,
        uint256 _endTime,
        bytes32 _eligibilityRoot
    ) {
        admin           = msg.sender;
        candidateIds    = _candidateIds;
        startTime       = _startTime;
        endTime         = _endTime;
        eligibilityRoot = _eligibilityRoot;
    }

    // ── vote() — called by voter's wallet via frontend ──────
    function vote(
        uint256 candidateId,
        bytes32[] calldata merkleProof
    ) external {
        require(block.timestamp >= startTime,  'Election not started');
        require(block.timestamp <= endTime,    'Election has ended');
        require(!hasVoted[msg.sender],         'Already voted');
        require(!finalized,                    'Election finalized');
        require(_validCandidate(candidateId),  'Invalid candidate');
        require(
            _verifyMerkle(msg.sender, merkleProof),
            'Not eligible to vote'
        );

        hasVoted[msg.sender]       = true;
        voteCounts[candidateId]   += 1;

        emit VoteCast(msg.sender, candidateId, block.timestamp);
    }

    // ── getResults() — read-only, no gas ────────────────────
    function getResults()
        external view
        returns (uint256[] memory ids, uint256[] memory counts)
    {
        ids    = candidateIds;
        counts = new uint256[](candidateIds.length);
        for (uint i = 0; i < candidateIds.length; i++) {
            counts[i] = voteCounts[candidateIds[i]];
        }
    }

    // ── finalize() — called by backend after RC approval ────
    function finalize() external {
        require(msg.sender == admin,           'Only admin');
```

```
        require(block.timestamp > endTime,     'Election not ended');
        finalized = true;
        emit ElectionFinalized(block.timestamp);
    }

    // ── Internal helpers ──────────────────────────────────────
    function _validCandidate(uint256 id) internal view returns (bool) {
        for (uint i = 0; i < candidateIds.length; i++) {
            if (candidateIds[i] == id) return true;
        }
        return false;
    }

    function _verifyMerkle(
        address voter,
        bytes32[] calldata proof
    ) internal view returns (bool) {
        bytes32 leaf = keccak256(abi.encodePacked(voter));
        bytes32 hash = leaf;
        for (uint i = 0; i < proof.length; i++) {
            if (hash <= proof[i]) {
                hash = keccak256(abi.encodePacked(hash, proof[i]));
            } else {
                hash = keccak256(abi.encodePacked(proof[i], hash));
            }
        }
        return hash == eligibilityRoot;
    }
}
```

3.2 What Each Require Statement Does

Condition	Category	Explanation
	Timing	Prevents voting before election opens. Enforced on-chain — cannot be bypassed.
	Timing	Prevents voting after election closes. Backend endTime and contract endTime must match.
	Anti-duplicate	This is the authoritative double-vote check (replaces the backend pre-check in Document 1).
	Lifecycle	Prevents votes after RC has finalized the results.
	Integrity	Ensures voter cannot pick a candidate ID that was not configured at deploy time.
	Eligibility	Validates the Merkle proof against the root stored at deploy. Ineligible voters are rejected.

Section 4 — Contract Deployment (Backend)

Contract deployment is performed by the backend using a platform wallet. This is an administrative action, not a voter action. The backend uses ethers.js with a private key stored in environment variables.

4.1 Hardhat Configuration

⚙️ **hardhat.config.js**

```
// hardhat.config.js
require('@nomicfoundation/hardhat-toolbox');
require('dotenv').config();

module.exports = {
  solidity: '0.8.20',
  networks: {
    hardhat: {},          // Local — used for development
    sepolia: {            // Testnet — used for final demo
      url:     process.env.ALCHEMY_SEPOLIA_URL,
      accounts:[process.env.PLATFORM_PRIVATE_KEY],
    },
  },
};
```

🔑 **.env**

```
# .env  (NEVER commit this file to Git)
ALCHEMY_SEPOLIA_URL=https://eth-sepolia.g.alchemy.com/v2/YOUR_API_KEY
PLATFORM_PRIVATE_KEY=0xYOUR_PLATFORM_WALLET_PRIVATE_KEY
```

4.2 Backend Deployment Service

This function is called when the Org Admin clicks Trigger Deployment in the dashboard. It deploys the contract and saves the address to the database.

🚀 **backend/services/deployElection.js**

```
// backend/services/deployElection.js
const { ethers } = require('ethers');
const ElectionArtifact =
require('../../artifacts/contracts/Election.sol/Election.json');

async function deployElectionContract(electionConfig) {
  const {
    candidateIds,    // Array of integers, e.g. [1, 2, 3]
    startTime,       // UNIX timestamp (number)
    endTime,         // UNIX timestamp (number)
    eligibilityRoot, // bytes32 Merkle root hex string
  } = electionConfig;
```

```
  // Connect to network via Alchemy RPC
  const provider = new ethers.JsonRpcProvider(process.env.ALCHEMY_SEPOLIA_URL);
  const wallet   = new ethers.Wallet(process.env.PLATFORM_PRIVATE_KEY, provider);

  // Deploy
  const factory  = new ethers.ContractFactory(
    ElectionArtifact.abi,
    ElectionArtifact.bytecode,
    wallet
  );
  const contract = await factory.deploy(
    candidateIds,
    startTime,
    endTime,
    eligibilityRoot
  );
  await contract.waitForDeployment();

  const contractAddress = await contract.getAddress();

  // Save to DB
  await db.query(
    `UPDATE elections SET contract_address = $1, status = 'SCHEDULED' WHERE id =
$2`,
    [contractAddress, electionConfig.electionId]
  );

  return contractAddress;
}
```

Section 5 — Frontend Voting Flow (Web3 Integration)

This is where Document 1's correction is most visible. The frontend wallet owns the signing process. The backend is not involved in submitting the vote transaction.

5.1 Wallet Connection Service

🔌 **frontend/services/web3.js**

```
// frontend/services/web3.js
import { ethers } from 'ethers';

// Connect MetaMask and return a signer
export async function connectWallet() {
  if (!window.ethereum) {
    throw new Error('MetaMask not installed. Please install it to vote.');
  }
  await window.ethereum.request({ method: 'eth_requestAccounts' });
  const provider = new ethers.BrowserProvider(window.ethereum);
  const signer   = await provider.getSigner();
```

```
  return { provider, signer };
}

// Get a contract instance connected to the voter's signer
export function getElectionContract(contractAddress, abi, signer) {
  return new ethers.Contract(contractAddress, abi, signer);
}
```

5.2 Vote Casting Function

frontend/services/castVote.js

```
// frontend/services/castVote.js
import { connectWallet, getElectionContract } from './web3';
import ElectionABI from '../../artifacts/contracts/Election.sol/Election.json';
import axios from 'axios';

export async function castVote(electionId, contractAddress, candidateId) {

  // Step 1 — Connect wallet
  const { signer } = await connectWallet();
  const voterAddress = await signer.getAddress();

  // Step 2 — Get Merkle proof from backend
  const { data: { merkleProof } } = await axios.get(
    `/api/elections/${electionId}/merkle-proof?voter=${voterAddress}`
  );

  // Step 3 — Get contract instance (connected to voter's wallet)
  const contract = getElectionContract(
    contractAddress,
    ElectionABI.abi,
    signer  // ← Voter's wallet signs the tx, NOT the backend
  );

  // Step 4 — Call vote() — MetaMask will popup asking voter to sign
  const tx = await contract.vote(candidateId, merkleProof);

  // Step 5 — Wait for on-chain confirmation
  const receipt = await tx.wait();
  const txHash  = receipt.hash;

  // Step 6 — Send tx hash to backend for recordkeeping ONLY
  await axios.post('/api/votes/confirm', {
    electionId,
    voterAddress,
    txHash,
    candidateId,
    timestamp: Date.now(),
  });

  return txHash; // Display on voter receipt screen
}
```

5.3 Backend Endpoint to Receive Tx Hash

This replaces Document 1's concept of the backend "monitoring" the blockchain. Instead, the backend passively receives the hash from the frontend.

📡 **backend/routes/votes.js**

```
// backend/routes/votes.js
router.post('/confirm', authenticate, async (req, res) => {
  const { electionId, voterAddress, txHash, candidateId, timestamp } = req.body;

  // Validate: ensure this voter is registered for this election
  const voter = await db.query(
    `SELECT id FROM voter_verifications
     WHERE election_id = $1 AND wallet_address = $2 AND status = 'VERIFIED'`,
    [electionId, voterAddress]
  );
  if (!voter.rows.length) return res.status(403).json({ error: 'Voter not verified'
});

  // Store tx hash — this is for audit trail and UI only
  await db.query(
    `INSERT INTO votes (election_id, voter_address, tx_hash, candidate_id,
voted_at)
     VALUES ($1, $2, $3, $4, to_timestamp($5 / 1000.0))`,
    [electionId, voterAddress, txHash, candidateId, timestamp]
  );

  res.json({ success: true, txHash });
});
```

Section 6 — Merkle Tree Implementation (Backend)

The Merkle tree is built in the backend from the finalized voter list. The root is stored in the smart contract at deployment. Individual proofs are generated on-demand when a voter initiates a vote.

6.1 Install the Merkle Tree Library

📦 **Installation**

```
npm install @openzeppelin/merkle-tree

# This library generates standard Merkle trees compatible with
# OpenZeppelin's on-chain MerkleProof.sol verifier.
# The contract in Section 3 uses a custom verifier — but the
# tree structure is the same.
```

6.2 Building the Root (Admin Locks Eligibility)

🌲 **backend/services/merkle.js**

```
// backend/services/merkle.js
const { StandardMerkleTree } = require('@openzeppelin/merkle-tree');

// Step 1 — Called when Admin locks the voter list
// walletAddresses: string[] — list of voter wallet addresses
function buildMerkleTree(walletAddresses) {
  const leaves = walletAddresses.map(addr => [addr]);
  const tree   = StandardMerkleTree.of(leaves, ['address']);
  return {
    root: tree.root,     // bytes32 hex — stored in contract
    tree,                // full tree — stored in DB as JSON
  };
}

// Step 2 — Called when a voter initiates a vote
// tree: the stored tree (parsed from DB JSON)
// voterAddress: the wallet address of the voting user
function getProofForVoter(tree, voterAddress) {
  for (const [i, v] of tree.entries()) {
    if (v[0].toLowerCase() === voterAddress.toLowerCase()) {
      return tree.getProof(i); // Returns bytes32[] array
    }
  }
  throw new Error('Voter address not found in eligibility tree');
}

module.exports = { buildMerkleTree, getProofForVoter };
```

6.3 API Endpoint — Serve Proof to Frontend

📡 **Merkle Proof API Endpoint**

```
// backend/routes/elections.js
router.get('/:id/merkle-proof', authenticate, async (req, res) => {
  const { id: electionId } = req.params;
  const voterAddress = req.query.voter;

  // Retrieve stored tree JSON from DB
  const result = await db.query(
    `SELECT merkle_tree_json FROM elections WHERE id = $1`,
    [electionId]
  );
  const tree = StandardMerkleTree.load(
    JSON.parse(result.rows[0].merkle_tree_json)
  );

  const proof = getProofForVoter(tree, voterAddress);
  res.json({ merkleProof: proof });
});
```

Section 7 — Reading Results (Free, No Gas)

Reading results from the smart contract is a view call — no transaction, no gas, no MetaMask popup. The frontend calls `getResults()` directly on the contract using a read-only provider.

7.1 Frontend — Fetch Results from Contract

📊 **frontend/services/getResults.js**

```
// frontend/services/getResults.js
import { ethers } from 'ethers';
import ElectionABI from '../../artifacts/contracts/Election.sol/Election.json';

export async function fetchResultsFromChain(contractAddress) {
  // Read-only provider — no wallet needed
  const provider = new ethers.JsonRpcProvider(
    process.env.REACT_APP_ALCHEMY_SEPOLIA_URL
  );

  const contract = new ethers.Contract(
    contractAddress,
    ElectionABI.abi,
    provider   // ← provider only, no signer
  );

  // view call — free, instant
  const [candidateIds, voteCounts] = await contract.getResults();

  return candidateIds.map((id, i) => ({
    candidateId: Number(id),
    votes:       Number(voteCounts[i]),
```

```
  }));
}
```

7.2 Result Committee — Dual Audit Query

The RC dashboard compares the blockchain tally (from `getResults()`) against the backend database count. Both numbers should match.

🔍 **backend/routes/audit.js**

```
// backend/routes/audit.js
router.get('/:id/audit', authenticateRC, async (req, res) => {
  const { id: electionId } = req.params;

  // 1. Fetch contract address from DB
  const election = await db.query(
    `SELECT contract_address FROM elections WHERE id = $1`,
    [electionId]
  );
  const contractAddress = election.rows[0].contract_address;

  // 2. Read on-chain tally (view call — free)
  const provider  = new ethers.JsonRpcProvider(process.env.ALCHEMY_SEPOLIA_URL);
  const contract  = new ethers.Contract(contractAddress, ElectionABI.abi,
provider);
  const [ids, onChainCounts] = await contract.getResults();

  // 3. Read DB tally
  const dbTally = await db.query(
    `SELECT candidate_id, COUNT(*) FROM votes
     WHERE election_id = $1 GROUP BY candidate_id`,
    [electionId]
  );

  // 4. Return both for RC comparison
  res.json({
    onChain: ids.map((id, i) => ({ candidateId: Number(id), count:
Number(onChainCounts[i]) })),
    database: dbTally.rows,
    match: /* comparison logic here */
  });
});
```

Section 8 — Gas Fee Strategy for College Project

8.1 Who Pays Gas and When

Operation	Payer	Notes
Contract Deployment	Backend (platform wallet)	Paid in test ETH (Sepolia). One-time per election. Most expensive operation.
Casting a Vote	Voter (their MetaMask wallet)	Each voter pays their own gas. Provide free Sepolia ETH via faucet for demo.
Reading Results	Nobody	View calls are free — no transaction submitted.
Checking hasVoted	Nobody	View call — free.
RC Finalize (optional)	Backend (platform wallet)	Only if you add a finalize() call. Small gas cost.

8.2 Development Phases

Phase 1 — Local Dev
Use Hardhat local network. All accounts pre-funded with 10,000 fake ETH. No real money, instant blocks. Run: `npx hardhat node`

Phase 2 — Final Demo
Deploy to Sepolia testnet. Get free test ETH from `sepoliafaucet.com` or `faucet.quicknode.com`. Distribute to demo voters beforehand.

8.3 Hardhat Local Development Commands

💻 **Terminal Commands**

```
# 1. Install dependencies
npm install --save-dev hardhat @nomicfoundation/hardhat-toolbox
npm install ethers dotenv @openzeppelin/merkle-tree

# 2. Start local blockchain (keep running in a separate terminal)
npx hardhat node

# 3. Compile contracts
npx hardhat compile

# 4. Run tests
npx hardhat test
```

```
# 5. Deploy to local network
npx hardhat run scripts/deploy.js --network hardhat

# 6. Deploy to Sepolia (for demo)
npx hardhat run scripts/deploy.js --network sepolia
```

💡 **Tip for Demo Day** Deploy one test election to Sepolia the day before your demo. Have 3–4 MetaMask accounts funded with Sepolia ETH ready to cast votes live. This is far more impressive than a local demo.

Section 9 — Implementation Checklist

Use this checklist to track progress. Items are ordered by dependency — complete them in sequence.

9.1 Smart Contract

☐	**Write Election.sol** — Constructor, vote(), getResults(), finalize(), Merkle verifier
☐	**Write unit tests** — Test: timing lock, double-vote rejection, invalid candidate, Merkle proof verification
☐	**Deploy to Hardhat local** — Confirm all tests pass on local network
☐	**Deploy to Sepolia** — Confirm deployment and store contract address

9.2 Backend

☐	**Add PLATFORM_PRIVATE_KEY to .env** — Generate a dedicated platform wallet in MetaMask
☐	**Build deployElection.js service** — Triggered when Admin clicks 'Trigger Deployment'
☐	**Build merkle.js service** — buildMerkleTree() and getProofForVoter()
☐	**Add GET /elections/:id/merkle-proof endpoint** — Serves proof to authenticated voter
☐	**Add POST /votes/confirm endpoint** — Receives Tx hash from frontend, stores in DB
☐	**Add GET /audit/:id endpoint** — Returns on-chain vs DB tally for RC dashboard
☐	**Store contract_address in elections table** — Add column if not already present

☐	**Store merkle_tree_json in elections table** — Needed to generate proofs on demand

9.3 Frontend

☐	**Install MetaMask in browser** — Required for all voting operations
☐	**Build web3.js service** — connectWallet() and getElectionContract()
☐	**Build castVote.js service** — Full flow: connect → get proof → sign → confirm → send hash
☐	**Update VotingPage UI** — 'Connect Wallet' button, candidate selection, MetaMask prompt, receipt screen
☐	**Build getResults.js service** — Read-only call to getResults() without wallet
☐	**Display Tx hash on receipt** — Link to Sepolia Etherscan for independent verification

9.4 Database Schema Additions

🗄️ **Database Migrations**

```
-- Add to elections table
ALTER TABLE elections ADD COLUMN contract_address VARCHAR(42);
ALTER TABLE elections ADD COLUMN merkle_root       CHAR(66);
ALTER TABLE elections ADD COLUMN merkle_tree_json  TEXT;

-- Add to votes table (replaces or supplements existing)
ALTER TABLE votes ADD COLUMN tx_hash       CHAR(66);
ALTER TABLE votes ADD COLUMN wallet_address VARCHAR(42);
```

Section 10 — Quick Reference Summary

Question	Answer	Notes
Who deploys the contract?	Backend (platform wallet)	Admin action — not voter-facing
Who signs the vote Tx?	Voter (MetaMask wallet)	Critical — see Section 1.1 correction
Who prevents double voting?	Smart contract (on-chain)	Critical — see Section 1.2 correction

Question	Answer	Notes
Who sends the Tx hash to backend?	Frontend (after vote)	Critical — see Section 1.3 correction
Who reads vote counts?	Frontend (view call)	Free, no gas, no wallet needed
Who builds the Merkle tree?	Backend (on eligibility lock)	Root goes into contract at deploy
Who generates Merkle proofs?	Backend API (on voter request)	Served to frontend just before vote
Who verifies the Merkle proof?	Smart contract	Done inside vote() on-chain
Where are results stored?	Blockchain (source of truth)	DB has a copy for UI convenience only
What testnet for demo?	Sepolia	Use Alchemy RPC + Sepolia faucet

✅ Key Principle to Remember

The blockchain protects the **act of voting**. The backend protects the **identity and eligibility** of voters. These two responsibilities must never be swapped. As long as every vote transaction is signed by the voter's own wallet, the integrity of the election is mathematically guaranteed — no administrator, developer, or hacker can alter the outcome without detection.